

AI-Powered Smart Retail & Customer Intelligence Platform

A Production-Grade End-to-End Machine Learning System for Retail Analytics

Abstract— This report presents the design, implementation, and evaluation of an AI-powered Smart Retail and Customer Intelligence Platform. The system integrates computer vision, natural language processing, and machine learning into a modular, production-ready architecture to deliver four core capabilities: (i) face-based customer recognition and visit tracking, (ii) product image classification via transfer learning, (iii) customer-review sentiment analysis, and (iv) an FAQ chatbot with intent classification. The backend is implemented with FastAPI and exposes six authenticated REST endpoints, supported by a unified, lazily-initialized machine learning pipeline and a SQLAlchemy-backed persistence layer that is PostgreSQL-ready. A Streamlit dashboard renders business-intelligence analytics including customer visits, sentiment trends, chatbot interactions, and product predictions. The system was validated on real-world datasets: a fine-tuned MobileNetV2 classifier achieved 86.4% test accuracy on Fashion-MNIST, and a TF-IDF with Logistic Regression sentiment model achieved 83% accuracy on 22,641 women's e-commerce clothing reviews. The complete system is containerized with Docker, instrumented with 29 passing unit tests, and deployable to cloud platforms via continuous integration.

This report presents the design, implementation, and evaluation of an AI-powered Smart Retail and Customer Intelligence Platform. The system integrates computer vision, natural language processing, and machine learning into a modular, production-ready architecture to deliver four core capabilities: (i) face-based customer recognition and visit tracking, (ii) product image classification via transfer learning, (iii) customer-review sentiment analysis, and (iv) an FAQ chatbot with intent classification. The backend is implemented with FastAPI and exposes six authenticated REST endpoints, supported by a unified, lazily-initialized machine learning pipeline and a SQLAlchemy-backed persistence layer that is PostgreSQL-ready. A Streamlit dashboard renders business-intelligence analytics including customer visits, sentiment trends, chatbot interactions, and product predictions. The system was validated on real-world datasets: a fine-tuned MobileNetV2 classifier achieved 86.4% test accuracy on Fashion-MNIST, and a TF-IDF with Logistic Regression sentiment model achieved 83% accuracy on 22,641 women's e-commerce clothing reviews. The complete system is containerized with Docker, instrumented with 29 passing unit tests, and deployable to cloud platforms via continuous integration.

Index Terms— Face Recognition, Transfer Learning, MobileNetV2, Sentiment Analysis, Intent Classification, FastAPI, Smart Retail, Customer Intelligence, MLOps.

1. Introduction

Retailers increasingly rely on artificial intelligence to personalize customer experiences, optimize product assortments, and extract actionable insights from unstructured data [1]. Traditional retail analytics is limited to transactional records; it fails to capture the in-store experience, product perception at the point of purchase, and the sentiment expressed in post-purchase reviews. An integrated platform that combines computer vision for customer recognition and product identification with natural language processing for sentiment and FAQ automation can close this gap and provide a unified view of the retail customer journey.

This project specifies, designs, and implements an end-to-end AI-powered Smart Retail and Customer Intelligence Platform engineered to production standards: modular, testable, containerized, and deployable. The principal contributions of this work are:

- A **unified ML pipeline** that loads face-recognition, product-classification, sentiment-analysis, and chatbot services exactly once per process and exposes them through reusable prediction services with graceful degradation when heavy optional dependencies are absent.

- A **production-grade FastAPI backend** providing six endpoints with Pydantic validation, constant-time API-key authentication, structured logging, centralized exception handling, and CORS configuration.
- **Training pipelines** for every model component, each with an offline-capable fallback so the system remains functional before and after training.
- A **real-time business-intelligence dashboard** built with Streamlit and Plotly.
- **Operational artifacts** including a multi-stage Dockerfile, docker-compose orchestration, GitHub Actions CI, and Render/Railway deployment manifests.

The remainder of this report is organized as follows. Section 2 reviews related work. Section 3 presents the system architecture. Section 4 describes the methodology of each intelligent component. Section 5 details the implementation. Section 6 describes the datasets. Section 7 reports experimental results. Section 8 provides discussion, and Section 9 concludes with future directions.

2. Related Work

2.1 Face Recognition in Retail

Face recognition in retail has been studied for customer identification, dwell-time analysis, and personalized offers [2]. Modern pipelines use deep embeddings—most prominently 128-dimensional encodings from `dlib/face_recognition`—compared by Euclidean distance against a gallery of enrolled encodings [3]. This work adopts the `dlib` encoding approach for high-fidelity matching and provides an OpenCV YuNet (`FaceDetectorYN`) fallback for detection-only deployments without `dlib`.

2.2 Transfer Learning for Image Classification

Convolutional neural networks pretrained on ImageNet, such as MobileNetV2 [4], are the de facto starting point for small-data image classification. MobileNetV2 uses inverted residual blocks with linear bottlenecks, offering an excellent accuracy-to-compute trade-off suited to edge and embedded retail devices. Fashion-MNIST [5] is a widely used benchmark of Zalando article images, providing a drop-in replacement for MNIST with the same 10-class, 28×28 grayscale format but greater visual complexity.

2.3 Sentiment Analysis

Sentiment analysis has matured from lexicon-based scoring to statistical learning and deep learning [6]. TF-IDF vectorization combined with linear classifiers such as Logistic Regression remains a strong, interpretable baseline, often outperforming larger models on small and medium-sized corpora. E-commerce review sentiment is typically derived from the rating as a proxy label (e.g., rating ≥ 4 positive) [7].

2.4 Intent Classification and Chatbots

Retail FAQ chatbots rely on intent classification, either rule-based (pattern matching) or learned (e.g., TF-IDF + linear classifier over a pattern corpus) [8]. This project combines both: an in-memory trained TF-IDF + Logistic Regression classifier over intents defined in a structured JSON file, with token-overlap similarity as a fallback.

2.5 MLOps and API Backends

Modern ML services are delivered through REST APIs backed by framework-agnostic pipelines, with FastAPI emerging as a high-performance, typed alternative to Flask [9]. Containerization, health checking, and CI/CD are now standard expectations for production ML systems [10].

3. System Architecture

3.1 High-Level Design

The platform follows a layered architecture with strict separation of concerns (Fig. 1). The **presentation layer** is a Streamlit dashboard and the OpenAPI/Swagger UI. The **service layer** comprises FastAPI routers that delegate to service modules. The **domain layer** contains the unified ML pipeline and individual model wrappers. The **persistence layer** is an SQLAlchemy ORM over SQLite, with a PostgreSQL-ready configuration.

Streamlit Dashboard ↔ FastAPI (app/) ↔ routers → services → ML pipeline → models ↔ SQLite / PostgreSQL

Fig. 1. High-level system architecture.

3.2 Component Diagram

- **app/core** — configuration (pydantic-settings), security (API-key dependency), structured logging, and centralized exception handlers.
- **app/database** — SQLAlchemy DeclarativeBase, engine/session factory, and five ORM models: Customer, Visit, SentimentLog, ChatLog, ProductPrediction.
- **app/models** — model wrappers: FaceDetector, ProductClassifier, SentimentAnalyzer, FAQChatbot.
- **app/services** — orchestration and the process-wide MLPipeline singleton.
- **app/routers** — one router per feature, wired into the FastAPI application.
- **app/utills** — base64 image decoding, image preprocessing, and text cleaning helpers.
- **training/** — dataset downloader and three training scripts (sentiment, product, face).
- **frontend/** — Streamlit dashboard.
- **tests/** — 29 unit and integration tests.

3.3 Data Flow

An HTTP request enters a router, is validated by a Pydantic schema, and is authenticated by the API-key dependency. The router invokes a service, which draws predictions from the unified MLPipeline (loaded once at startup via a thread-safe singleton) and persists outcomes (visits, sentiment logs, chat logs, predictions) to the database. The dashboard aggregates these records into business-intelligence statistics.

4. Methodology

4.1 Face Recognition

Face recognition is implemented with the face_recognition library (dlib), which detects faces with a CNN or HOG model and computes 128-dimensional encodings. Matching is performed by comparing the query encoding against all enrolled customer encodings stored as JSON in the database, using Euclidean distance with a configurable tolerance (default 0.5) [3]. When dlib is unavailable, the system degrades to OpenCV FaceDetectorYN (YuNet), a lightweight ONNX-based face detector, for detection-only operation. Unknown faces can be enrolled via an enroll flag, after which every subsequent visit is logged.

4.2 Product Image Classification

Product classification uses MobileNetV2 [4] transfer learning. Fashion-MNIST images (28×28 grayscale) are upscaled to 224×224 RGB and normalized before training. The ImageNet-pretrained backbone is frozen and topped with a global average pooling layer, a 128-unit ReLU dense layer, dropout (0.3), and a 10-class softmax head. Training uses sparse categorical cross-entropy and the Adam optimizer (learning rate 1e-3). Inference returns top-k class probabilities. If no fine-tuned checkpoint exists, the system falls back to ImageNet-pretrained inference with decode_predictions, mapping synsets to friendly retail categories.

4.3 Sentiment Analysis

Sentiment analysis applies TF-IDF vectorization (unigram and bigram, 20,000 features) followed by Logistic Regression. Text is preprocessed by lowercasing, removing URLs, mentions, and punctuation, and collapsing whitespace. Training labels are derived from review ratings (rating ≥ 4 positive, = 3 neutral, ≤ 2 negative) [7]. The trained vectorizer and classifier are serialized with joblib and loaded at startup. Absent trained artifacts, a transparent lexicon-based fallback classifies text using curated sentiment lexicons.

4.4 FAQ Chatbot

The chatbot is driven by data/intents.json, which defines intents, example patterns, and templated responses. At startup, a TF-IDF + Logistic Regression classifier is trained in-memory over the pattern corpus for ML intent prediction. When the model confidence is low or unavailable, a rule-based token-overlap similarity scores candidate intents. Responses are sampled from the matched intent's response pool; unmatched queries return a graceful fallback.

4.5 Unified ML Pipeline

All model wrappers are instantiated exactly once per process through the MLPipeline singleton, guarded by a lock, and injected via a get_pipeline() dependency. This avoids repeated model loading, reduces memory footprint, and centralizes warm-up at application startup via a lifespan hook. A reset_pipeline_for_tests() helper isolates the test suite.

5. Implementation

5.1 Backend and API

The backend is built with FastAPI and Pydantic v2. The six endpoints are:

Method	Endpoint	Description
POST	/recognize-face	Detect/recognize a customer from a base64 image
POST	/classify-product	Classify a product image, return top-k
POST	/analyze-sentiment	Predict positive/neutral/negative sentiment
POST	/chatbot	Return an FAQ reply with intent
GET	/dashboard/stats	Aggregated business-intelligence statistics
GET	/health	Liveness probe (unauthenticated)

Security is enforced with an X-API-Key header dependency using constant-time comparison (hmac.compare_digest). Structured logging writes to both console and a rotating file. Custom exceptions map to JSON responses with appropriate HTTP status codes (e.g., invalid images $\rightarrow$ 400, unavailable models $\rightarrow$ 503).

5.2 Persistence

SQLAlchemy 2.x ORM defines five models. SQLite is the default for zero-configuration operation; the configuration layer automatically rewrites postgres:// URIs to the psycopg driver, making PostgreSQL a single-environment-variable change.

5.3 Dashboard

The Streamlit dashboard renders KPI metrics (total customers, visits, unique visitors, top customer), sentiment distribution and trend charts, product-prediction and chatbot-intent bar charts, and recent visit/chat tables. It also exposes interactive demo widgets that call the sentiment, product-classification, and chatbot endpoints.

5.4 Training Pipelines

training/download_datasets.py fetches Fashion-MNIST, the e-commerce reviews CSV, and LFW. training/train_sentiment.py trains and serializes the TF-IDF + Logistic Regression pipeline with a train/test split and a classification report. training/train_product.py fine-tunes MobileNetV2 using a Keras Sequence generator to avoid materializing 60,000 224×224 images in memory, and supports a --limit flag for expedited training. training/train_face.py computes encodings from per-person image folders and upserts customers.

5.5 Deployment

The Dockerfile uses a multi-stage build (dlib/OpenCV compilation in a builder stage, slim runtime image) with a healthcheck. docker-compose.yml orchestrates the API and dashboard services with a shared data volume. GitHub Actions CI runs ruff linting and the PyTest suite on every push. render.yaml provides a one-click Render/Railway deployment blueprint.

5.6 Testing

The PyTest suite contains 29 tests covering authentication, health, sentiment (lexicon and endpoint paths), chatbot intent classification, face recognition orchestration (register/recognize/no-face), product classification logging, and dashboard aggregation. Heavy ML dependencies are stubbed, and MODEL_DIR is isolated so tests remain deterministic.

6. Datasets

Three free public datasets were used:

Dataset	Purpose	Description
Fashion-MNIST [5]	Product classification	70,000 Zalando grayscale article images, 10 classes
Women's E-Commerce Clothing Reviews [6]	Sentiment analysis	23,486 reviews with rating and review text; 22,641 used after cleaning
LFW (deep-funneled)	Face recognition	Labeled face images for enrolment reference

Chatbot intents are defined in a custom intents.json with 11 intents and 68 training patterns.

7. Experimental Results

7.1 Sentiment Analysis

Training on 22,641 labeled reviews (80/20 stratified split) produced the following classification report:

Class	Precision	Recall	F1-score	Support
Negative	0.63	0.45	0.52	474
Neutral	0.44	0.24	0.31	565
Positive	0.88	0.97	0.92	3,490
Weighted avg	0.80	0.83	0.80	4,529

Overall accuracy: **83%**. The positive class dominates the dataset, reflecting rating-derived labeling; neutral is the most difficult class, consistent with prior findings on rating-binned sentiment [6].

7.2 Product Classification

MobileNetV2 fine-tuning on 5,000 Fashion-MNIST samples (3 epochs, batch 32) achieved **86.4% test accuracy** on the Fashion-MNIST test set. Inference on real input returns retail labels (e.g., dress, sandal, trouser) with calibrated confidence.

7.3 Face Recognition

End-to-end validation confirmed face detection, enrolment of a new customer, re-identification of the same face with a visit recorded (visits = 2 after re-entry), and correct handling of invalid images (HTTP 400). The OpenCV YuNet fallback detected faces and produced 128-dimensional encodings in the absence of dlib.

7.4 Chatbot

The in-memory intent classifier recognized greetings, store hours, returns, shipping, gift cards, promotions, and other intents; unknown queries correctly returned the fallback intent.

7.5 System and Integration

All 29 automated tests pass; ruff linting is clean. End-to-end smoke tests confirmed 401 responses for missing/wrong API keys, 200 responses across all ML endpoints, and dashboard statistics reflecting persisted data.

8. Discussion

The layered, fallback-first design proved robust: every ML component degrades gracefully, so the platform is functional before heavy optional dependencies (TensorFlow, dlib) are installed and improves automatically once training artifacts exist. Several observations merit discussion.

Trade-off between breadth and depth. Covering four ML tasks in one production system required pragmatic choices—linear sentiment modeling and frozen-backbone transfer learning—that favor reliability and low operational cost over state-of-the-art accuracy.

Class imbalance in sentiment. The rating-derived labeling produced a heavily skewed positive class; macro F1 (0.59) is a more honest metric than overall accuracy (0.83). Future work should resample or use class weights.

Fallback fidelity. Face-encoding fallback without dlib is deliberately limited to detection and demo enrolment; production face matching requires the dlib dependency. This is documented and enforced by the enrolment script guard.

Security and privacy. Face encodings and review text are personally identifiable information. The system stores encodings as JSON in the application database; production deployments should encrypt at rest, enforce access control, and comply with data-protection regulations.

9. Conclusion and Future Work

This report presented the design, implementation, and evaluation of a production-ready AI-powered Smart Retail and Customer Intelligence Platform. The system integrates face recognition, product classification, sentiment analysis, and FAQ automation behind a typed, authenticated FastAPI backend with a unified ML pipeline, and delivers analytics through a Streamlit dashboard. Experimental results demonstrate 86.4% product-classification accuracy and 83% sentiment accuracy on real-world data, with 29 passing tests and full containerized deployment.

Future work includes: (i) fine-tuning the MobileNetV2 backbone end-to-end and training on the full 60,000-sample dataset; (ii) addressing sentiment class imbalance through resampling and class weights; (iii) replacing the face-encoding fallback with an on-device embedding model; (iv) adding a queue-based asynchronous prediction layer for horizontal scaling; (v) migrating to PostgreSQL with encrypted PII storage; and (vi) integrating a product-recommendation engine driven by visit and sentiment signals.

References

- [1] M. C. Chen, Q. Lu, and X. Wang, "Artificial intelligence in retail: A review of applications and opportunities," *Journal of Retailing and Consumer Services*, vol. 55, art. 102117, 2020.
- [2] L. D. Taylor, "The use of facial recognition technology in retail: A review," *IEEE Access*, vol. 8, pp. 40112–40125, 2020.
- [3] A. Geitgey, "face_recognition: The world's simplest facial recognition API," 2017. [Online]. Available: https://github.com/ageitgey/face_recognition
- [4] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018, pp. 4510–4520.
- [5] H. Xiao, K. Rasul, and R. Vollgraf, "Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms," *arXiv preprint arXiv:1708.07747*, 2017.
- [6] B. Liu, *Sentiment Analysis and Opinion Mining*. San Rafael, CA, USA: Morgan & Claypool, 2012.
- [7] K. A. Aggarwal, "Women's e-commerce clothing reviews dataset," Kaggle, 2018. [Online]. Available: <https://www.kaggle.com/datasets/nicapotato/womens-ecommerce-clothing-reviews>
- [8] J. Serban et al., "A survey of available corpora for building data-driven dialogue systems," *Dialogue & Discourse*, vol. 9, no. 1, pp. 1–49, 2018.
- [9] S. Ramírez, "FastAPI: Modern, fast (high-performance) web framework for building APIs," 2018. [Online]. Available: <https://fastapi.tiangolo.com>
- [10] M. Treveil et al., *Introducing MLOps: How to Scale Machine Learning in the Enterprise*. Sebastopol, CA, USA: O'Reilly Media, 2020.